Assignment 6

Basic Info:

I used Claude and Copilot which I accessed on my computer.

Prompt:

Write the solutions for each puzzle file to a single file named solutions.txt.
• For each puzzle, the solutions.txt file should include the following:
o A single line: “Puzzle: x”, where x is the puzzle file name (e.g., puzzle1.txt).
o If no solutions are found, a single line: “No solution found.”
o If solutions are found, it should include all solutions followed by a single line: “Solutions found: n”, where n is the number of solutions found.
o The solutions should be in the same format as the input files: puzzle1.txt – puzzle5.txt.
• You must modify the Assignment 5 you receive, not your Assignment 5.
• Do not replace the entire file with your code. Your GTA will use the Github Blame button to check your modifications.
• Add your name to the Profile Comments as “Revised by:” along with a revision date.
• You must get all other required functionality from Assignment 5 working, even if the student you inherited it from did not.

Code:

Claude:

```
static void printGrid(std::ostream& out, const Grid& g) { // takes output stream and grid
    // MODIFIED (Assignment 6): Removed the divider lines so the output matches the
    // input puzzle file format (rows of 9 cells, '_' for blank)
    for (int r = 0; r < N; ++r) {
        for (int c = 0; c < N; ++c) {
            // prints '_' for empty cell overwise turns int into char
            out << (g[r][c] == 0 ? '_' : char('0' + g[r][c]));
            out << (c < N - 1 ? " " : "");
        }
```

```
            out << '\n';
        }
        out << '\n';
    }
    // MODIFIED (Assignment 6): Single output file for all puzzles
    std::ofstream outFile("solutions.txt");
    if (!outFile) {
        std::cerr << "Could not open solutions.txt for writing.\n";
        return 1;
    }

    for (size_t i = 0; i < files.size(); ++i) {
        const auto& name = files[i];

        // MODIFIED (Assignment 6): Blank line between puzzles, then the puzzle label
        if (i > 0) outFile << "\n";
        outFile << "Puzzle: " << name << "\n";

        SudokuPuzzle puzzle(name);
```

Copilot:

```
Puzzle: puzzle1.txt
std::ofstream outFile("solutions.txt");
if (!outFile)
{
    return 1;
}
outFile << "Puzzle: " << name << "\n";
```

```
if (!puzzle.load() || !puzzle.isInitialGridValid()) {
    outFile << "No solution found.\n\n";
    continue;
}
if (found == 0) {
    outFile << "No solution found.\n";
}
else {
    outFile << "Solutions found: " << found << "\n";
}
outFile << "\n";
```

Analysis:

In terms of correctness, both solve the puzzles correctly. They both implement the same solving logic. When it comes to execution time, they're the same. Since they use the same solving logic which is the bulk of the program neither's time differs. The same is true for space complexity. Since the main solver is the same the differences in the functions space and time are the same. The recursive calls are what would affect them and since this hasn't been modified, they are the same. The difference is mostly in maintainability. However, even with their differences they are quite similar. They both include essential comments and explanations of function operations.

Revisions:

I will use Claude for this assignment as I like the way it's organized and prints the puzzles more. I will improve executions time by tracking which numbers are used in each row, column, and block when picking the starting cell. This reduces the amount of checking the recursion does and makes the checking process faster. Thus, the time needed to solver more difficult puzzles is reduced. I will also have the program stop checking when it finds a cell with no other possible candidates.